03

MGU BCA - 3RD SEM

Design and Analysis of Algorithms

Module 3 - Dynamic Programming, Graphs, Backtracking

By Aben Sojan

Exam - 12 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

TOPIC 1

Dynamic Programming: General Concept

1

DP solves problems with overlapping subproblems and optimal substructure by solving each subproblem once and storing the answer.

Two styles: memoization (top-down recursion plus cache) and tabulation (bottom-up table).

Classic example: Fibonacci.

Naive recursion is O(2^n); DP makes it O(n).

Steps: define subproblems, write recurrence, compute bottom-up (or memoized), read answer.

QUESTION BANK

1 **Explain dynamic programming. How is it different from divide and conquer?**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Dynamic programming solves problems whose subproblems overlap: each distinct subproblem is solved once, stored, and reused instead of being recomputed.

It needs two properties: overlapping subproblems and optimal substructure (an optimum built from optima of subproblems).

Classic example: Fibonacci. fib(5) needs fib(4) and fib(3); fib(4) needs fib(3) again - the same fib(3) is recomputed many times.

Naive recursion explodes to O(2^n); DP stores each fib(k) once, dropping the time to O(n).

Difference from divide and conquer: D&C is for INDEPENDENT subproblems solved separately (merge sort's halves never repeat); DP is for OVERLAPPING subproblems solved once and shared.

Merge sort has no overlap, so DP would not help it; Fibonacci is almost all overlap, so D&C-style recursion is disastrous there.

Two implementation styles: memoization (top-down recursion plus a cache) and tabulation (bottom-up table).

Decision rule: overlap means DP, independent means divide and conquer.

2 Differentiate memoization and tabulation.

IMPORTANT

Both implement dynamic programming; they differ in direction and in which subproblems get computed.

Memoization is top-down: write the natural recursion, but cache every result so a repeated call returns the stored answer.

```
Pseudocode: if n in memo: return memo[n]; memo[n] = solve(n-1) + solve(n-2); return memo[n]
```

Tabulation is bottom-up: fill a table iteratively from the smallest subproblem upward, with no recursion at all.

```
Pseudocode: table[0] = 0; table[1] = 1; for i in 2..n: table[i] = table[i-1] + table[i-2]
```

Memoization computes only the subproblems actually needed and keeps the recursive structure; tabulation computes everything in order and avoids recursion overhead and stack limits.

Both give O(n) time and O(n) space for Fibonacci.

Memoization is lazier and more intuitive; tabulation is iterative and stack-safe - same DP idea, two flavours.

3 What are the two properties a problem needs for DP?

IMPORTANT

A problem suits dynamic programming only if it has two properties.

Overlapping subproblems: the same smaller problems appear again and again during recursion, so storing answers saves real work.

Without overlap there is nothing to reuse, and plain divide and conquer is the right tool instead.

Optimal substructure: an optimal answer to the whole problem can be constructed from optimal answers to its subproblems.

Without it, combining subproblem answers may not give the global optimum, and DP would produce wrong results.

Example with both: shortest path - the shortest S-to-T path through X combines the shortest S-to-X and X-to-T paths, and subpaths repeat across queries.

Counter-example: merge sort's halves are distinct every time, so it has no overlap and DP adds nothing.

Check both properties before reaching for DP; missing either one means a different technique

fits better.

TOPIC 2

Floyd Warshall Algorithm

2

Floyd Warshall finds shortest paths between ALL pairs of vertices: all-pairs shortest path.

DP idea: D[k][i][j] = shortest i-to-j path using only vertices 1..k as intermediates.

Recurrence: D[k][i][j] = min(D[k-1][i][j], D[k-1][i][k] + D[k-1][k][j]).

O(V^3) time; handles negative weights but not negative cycles.

QUESTION BANK

4 Explain the Floyd Warshall algorithm with its recurrence.

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Floyd Warshall computes the shortest path between EVERY pair of vertices: the all-pairs shortest path problem.

DP idea: let D[k][i][j] be the shortest distance from i to j using only vertices 1..k as intermediate stops.

Recurrence: D[k][i][j] = min(D[k-1][i][j], D[k-1][i][k] + D[k-1][k][j]).

Meaning: the best i-to-j path either ignores vertex k entirely, or goes i -> ... -> k -> ... -> j.

Base: D[0] holds direct edge weights, 0 on the diagonal, infinity where no edge exists.

Pseudocode:

```
initialise D with edge weights
for k in 1..V:
for i in 1..V:
for j in 1..V:
D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Only two layers are needed in practice, so the space is O(V^2).

Time is O(V^3) from the three nested loops - the price of computing all pairs.

It handles negative edge weights correctly, but not negative cycles, where distances would keep shrinking forever.

Three loops and one recurrence solve all-pairs shortest paths - the simplest O(V^3) algorithm in the syllabus.

5 Trace Floyd Warshall on a small graph.

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Graph: 1->2 weight 3, 2->3 weight 2, 1->3 weight 10.

Initial D[0]: row1 [0, 3, 10], row2 [inf, 0, 2], row3 [inf, inf, 0]. k = 1: allow vertex 1 as an intermediate.

No pair improves, since nothing useful routes through vertex 1 yet. k = 2: allow vertex 2.

Check D[1][3]: min(10, D[1][1][2] + D[1][2][3]) = min(10, 3 + 2) = 5.

The path 1->2->3 beats the direct edge. k = 3: allow vertex 3.

No further improvement is possible.

Final distances: 1->2 = 3, 2->3 = 2, 1->3 = 5 via vertex 2.

Each k either ignores the new vertex or routes through it - that binary choice is the whole recurrence in action.

The trace shows the DP table improving exactly when a useful intermediate appears.

6 Compare Floyd Warshall with Dijkstra.

IMPORTANT

Scope: Floyd Warshall solves all-pairs shortest paths in one run; Dijkstra solves single-source, so it must be run V times for all pairs.

Weights: Floyd Warshall tolerates negative edge weights; Dijkstra requires non-negative weights.

Time: Floyd Warshall is O(V^3); one Dijkstra is O(E log V), so V runs cost O(V * E log V).

On dense graphs, V runs of Dijkstra approach O(V^3 log V), making Floyd Warshall competitive or better.

Code simplicity: Floyd Warshall is three nested loops; Dijkstra needs a priority queue plus settled/relaxation bookkeeping.

Memory: Floyd Warshall keeps a V x V table, O(V^2); Dijkstra keeps O(V) distances per run.

Rule of thumb: all pairs on a dense graph, or any negative edges - Floyd Warshall; single source with non-negative weights - Dijkstra.

They answer different questions, so the exam tests whether you match the algorithm to the problem.

TOPIC 3

3

Breadth First Search (BFS)

BFS explores a graph level by level using a QUEUE: visit neighbours before going deeper.

Start at source, mark visited, enqueue; dequeue a vertex, enqueue its unvisited neighbours.

Time O(V + E); gives shortest path (fewest edges) in unweighted graphs.

QUESTION BANK

7 Explain BFS with an example. What is its complexity?

REPEATED (Oct 2021, Feb 2025)

Breadth First Search explores a graph level by level: all neighbours of the source first, then neighbours of neighbours, using a QUEUE.

Pseudocode:

```
queue = [source]; visited[source] = True
while queue:
u = queue.pop(0)
for v in neighbours(u):
if not visited[v]: visited[v] = True; queue.append(v)
```

Trace: S connected to A and B; A connected to C.

Queue starts [S].

`Pop S, enqueue A, B -> queue [A, B]. Pop A, enqueue C -> [B, C]. Pop B -> [C]. Pop C -> [].`

Visit order: S, A, B, C - level 0, then level 1, then level 2.

Complexity: each vertex is enqueued once and each edge examined once, giving O(V + E) with an adjacency list.

The queue is the key: first-in-first-out order is what enforces the level-by-level discipline.

BFS is the natural way to explore outward evenly, which is why it finds shortest paths in unweighted graphs.

8 Why does BFS give the shortest path in unweighted graphs?

IMPORTANT

In an unweighted graph, the shortest path means the fewest edges.

The queue processes vertices strictly in order of their distance from the source: all distance-1 vertices before any distance-2 vertex, and so on.

Therefore the first time BFS reaches a vertex, it has arrived via the smallest possible number of edges - no shorter route can exist.

Recording each vertex's parent at discovery time traces one such shortest path back to the source.

With weighted edges this breaks down, because a two-edge path can be shorter than a one-edge path; Dijkstra is needed there.

FIFO order plus unit weights equals shortest paths - remove either ingredient and the guarantee vanishes.

9 What are the applications of BFS?

IMPORTANT

Shortest path in unweighted graphs: fewest hops between two nodes, e.g. degrees of separation in a social network.

Connected components: one BFS from each unvisited vertex labels every component of the graph.

Bipartiteness test: colour levels alternately; an edge joining two same-colour vertices proves the graph is not bipartite.

Web crawlers: expand outward from seed pages level by level, which is BFS over the link graph.

Shortest path in mazes and grids: each step costs one, so BFS finds the quickest route.

Peer-to-peer and broadcast networks use BFS-like flooding to reach all nodes evenly.

Whenever fewest steps or level order matters, BFS is the tool.

TOPIC 4

Depth First Search (DFS)

4

DFS goes as deep as possible along each branch before backtracking, using a STACK or recursion.

Visit a vertex, recursively visit each unvisited neighbour.

Time O(V + E); uses discovery and finish times, useful for cycle detection and topological sort.

QUESTION BANK

10 Explain DFS with an example. What is its complexity?

REPEATED (Oct 2021, Feb 2025)

Depth First Search explores a graph by going as deep as possible along each branch before backtracking, using recursion as an implicit stack.

Pseudocode:

```
dfs(u):
visited[u] = True
for v in neighbours(u):
if not visited[v]: dfs(v)
```

Trace: edges S-A, S-B, A-C, starting at S.

```
Visit S; first neighbour A: visit A; A's neighbour C: visit C. C has no
unvisited neighbours: backtrack to A, then to S.
Next neighbour B: visit B. Done. Visit order: S, A, C, B.
```

Complexity: each vertex is visited once and each edge examined once, giving O(V + E) with an adjacency list.

The recursion stack records the current path, which is why DFS is natural for backtracking-style problems.

DFS trades BFS's level order for depth, which unlocks cycle detection and topological sorting.

11 Compare BFS and DFS.

REPEATED (Oct 2021, Feb 2025)

Data structure: BFS uses a queue (FIFO); DFS uses a stack or recursion (LIFO).

Order: BFS visits level by level outward; DFS dives deep down one branch before backtracking.

Shortest paths: BFS finds fewest-edge paths in unweighted graphs; DFS gives no such guarantee.

Memory: BFS stores a whole frontier level, O(V) in the worst case; DFS stores one path, O(depth), usually smaller.

Applications: BFS for shortest paths, components and bipartiteness; DFS for cycle detection, topological sort and backtracking search.

Both run in O(V + E) on adjacency lists and visit every reachable vertex exactly once.

Same complexity, opposite exploration order - the exam loves this contrast.

12 What are discovery and finish times in DFS?

IMPORTANT

DFS stamps every vertex twice: discovery time when it is first visited, finish time when all its neighbours are done.

Colour view: white means undiscovered, gray means discovered but still on the recursion stack, black means finished.

Discovery time marks entry, when the vertex turns gray; finish time marks exit, when it turns black.

The intervals nest properly: if v is discovered while u is gray, v's interval sits inside u's, proving v is a descendant of u in the DFS tree.

These times classify edges in directed graphs: a back edge to a gray ancestor reveals a cycle.

Sorting vertices by decreasing finish time yields a topological order of a DAG.

The two timestamps turn a simple walk into a full structural analysis of the graph.

TOPIC 5

Connected Components

5

A connected component is a maximal set of vertices all reachable from each other.

Count components: run DFS or BFS from every unvisited vertex; each launch is one component.

A graph is connected if it has exactly one component.

QUESTION BANK

13 How do you find connected components of a graph?

IMPORTANT

A connected component is a maximal group of vertices all reachable from each other; maximal means no vertex can be added to it.

Algorithm: keep a visited array and a counter starting at 0.

```
for each vertex v:
if not visited[v]:
run BFS or DFS from v # marks exactly its component
count += 1
```

Each launch explores precisely one component, because the search cannot cross over to unreachable vertices.

When the loop ends, count equals the number of components and every vertex is labelled with

its component.

Complexity is O(V + E), since each vertex and edge is processed once across all launches.

Counting launches of a graph search is the standard components algorithm.

14 How do you test whether a graph is connected?

IMPORTANT

A graph is connected if it has exactly one component, i.e. every vertex is reachable from every other vertex.

Run a single BFS or DFS from any vertex, say vertex 0, and count how many vertices get visited.

If the count equals V, all vertices were reached and the graph is connected.

If fewer than V were visited, some vertices are unreachable and the graph is disconnected.

This is the components algorithm stopped after the first launch, still costing O(V + E).

One search, one count, one answer.

TOPIC 6

DFS of a Directed Graph

6

In directed graphs DFS classifies edges: tree, back, forward, cross.

Back edge (to an ancestor) means the directed graph HAS a cycle.

Finishing times in decreasing order give a topological sort of a DAG.

QUESTION BANK

15 Explain the types of edges in DFS of a directed graph.

IMPORTANT

DFS of a directed graph classifies every edge by the state of its target vertex at the moment it is explored.

Tree edge: points to a white (undiscovered) vertex; it becomes part of the DFS tree.

Back edge: points to a gray vertex, i.e. an ancestor on the current recursion stack; its presence proves a directed cycle exists.

Forward edge: points to a black descendant, skipping down levels of the DFS tree.

Cross edge: points to a black vertex that is neither ancestor nor descendant, typically sitting in another branch or tree.

In undirected graphs only tree and back edges exist, which is why cycle detection there is

simpler.

Edge colours are read off the discovery and finish times, turning DFS into a cycle detector.

16 How does DFS detect a cycle in a directed graph?

IMPORTANT

A directed graph has a cycle if and only if DFS encounters a back edge - an edge to a vertex currently on the recursion stack.

Colour scheme: white means unvisited, gray means on the current DFS path, black means fully explored.

Pseudocode:

```
dfs(u):
colour[u] = gray
for v in neighbours(u):
if colour[v] == gray: cycle found -> return True
if colour[v] == white and dfs(v): return True
colour[u] = black; return False
```

Trace: edges A->B, B->C, C->A. dfs(A) grays A; dfs(B) grays B; dfs(C) sees neighbour A which is gray - back edge, cycle confirmed.

Without the gray distinction, the edge C->A would look like an ordinary visited edge and the cycle would be missed.

Complexity is O(V + E).

For undirected graphs the rule simplifies to: a visited neighbour that is not the parent means a cycle.

Gray means my ancestor - an edge back to one is a cycle, always.

17 What is topological sorting? How is it done with DFS?

IMPORTANT

A topological ordering of a DAG lists vertices so that every directed edge u->v has u before v; it models task scheduling with prerequisites.

It exists only for directed acyclic graphs - a cycle makes ordering impossible, since each vertex would have to precede itself.

DFS method: run DFS over the whole graph; whenever a vertex finishes (turns black), push it onto a stack.

```
After DFS completes, pop the stack: vertices emerge in decreasing order of
```

```
finish time.
```

Why it works: for any edge u->v, v finishes before u (v is explored from u), so u is pushed later and popped earlier.

Example: edges A->B, A->C, B->D, C->D.

Valid orders include A, B, C, D and A, C, B, D.

Complexity is O(V + E).

Finish time reversed is the one-line recipe for topological sort.

TOPIC 7

7

Backtracking: Concept

Backtracking builds candidates step by step and abandons a path as soon as it cannot lead to a solution.

It explores a state space tree; a bounding function prunes hopeless branches.

A node is promising if a solution might still lie below it; else backtrack to the parent.

Examples: N-Queens, sum of subsets, Hamiltonian circuit, graph colouring.

QUESTION BANK

18 Explain the backtracking technique with an example.

REPEATED (Feb 2025, Oct 2025 H)

Backtracking solves constraint problems by building candidates one choice at a time, abandoning a partial candidate the moment it cannot lead to a valid answer.

Think of a maze: walk forward; at a dead end, walk back to the last junction and try another turn - the walking back is the backtrack.

The choices form a state space tree: each node is a partial answer and its children add one more choice.

A bounding function prunes branches that cannot succeed, so huge parts of the tree are never explored.

Example: N-Queens places queens row by row; the moment a queen is attacked, that placement is undone and the next column is tried.

Pseudocode shape:

```
solve(partial):
if partial is complete: record it; return
for each legal next choice:
```

```
extend partial; solve(partial); undo the extension # backtrack
```

Backtracking is organised trial and error: try, test early, undo fast.

19 What is a state space tree? What is pruning?

IMPORTANT

The state space tree is the tree of all partial candidates: the root is the empty answer, and each child adds one more choice.

A complete root-to-leaf path is one full candidate; internal nodes are the prefixes tried along the way.

Pruning means cutting off a subtree without exploring it, using a bounding function that proves no solution lies inside.

A node is called promising if the partial answer could still extend to a solution; only promising nodes are expanded.

Example: in sum of subsets with target 5, any branch whose partial sum already exceeds 5 is pruned immediately.

Good pruning is what separates backtracking from blind brute force - it can shrink an exponential search to something practical.

The tree organises the search; pruning keeps it affordable.

20 Differentiate backtracking from brute force.

IMPORTANT

Brute force generates every complete candidate first and only then tests whether it satisfies the constraints.

Backtracking tests partial candidates during construction and abandons a branch as soon as it is doomed.

Example: 4-queens by brute force would test all 4^4 = 256 full placements; backtracking kills most branches after one or two rows.

Both are exponential in the worst case, since a problem may genuinely need the whole tree explored.

But in practice backtracking with strong pruning runs far faster, because dead branches die young.

Memory also favours backtracking: it needs only the current path, O(depth), while brute force may hold many candidates.

Brute force checks everything at the end; backtracking checks early and quits early.

TOPIC 8

8

N-Queens Problem

Place N queens on an N x N board so no two attack each other: different rows, columns and diagonals.

Place queens row by row; for each row try columns left to right, skipping attacked squares.

If no column works, backtrack: move the previous queen to its next column.

QUESTION BANK

21 **Explain the N-Queens problem and its backtracking solution.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The N-Queens problem asks for placements of N queens on an N x N board so that no two queens attack each other.

Attack rule: no two queens may share a row, a column, or a diagonal.

Backtracking strategy: place queens row by row from top to bottom, so rows are automatically distinct.

Pseudocode:

```
place(row):
if row == N: record the board; return
for col in 0..N-1:
if square (row, col) is safe:
put queen; place(row+1); remove queen # backtrack
```

Safety test against earlier queens at (r, c): col != c for the column, and abs(row - r) != abs(col - c) for the diagonals.

If no column is safe in a row, the recursion returns and the previous queen moves to its next column - that is the backtrack step.

Results: 4-queens has 2 solutions and 8-queens has 92; the worst case is exponential but pruning makes it practical.

One queen per row plus the attack test turns an impossible search into a guided one.

22 How do you test whether a square is under attack?

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Since queens are placed top to bottom, only earlier rows can threaten the new square.

Column test: the new square's column must differ from every earlier queen's column.

Diagonal test: for each earlier queen at (r, c), the new square (row, col) is attacked if abs(row - r) == abs(col - c).

The absolute-difference trick works because diagonals have slope 1: equal row-gap and column-gap means the same diagonal.

If the column test and both diagonal directions pass, the square is safe to use.

Example: queens at (0,1) and (1,3); square (2,0): columns differ; |2-0|=2 vs |0-1|=1 fine; |2-1|=1 vs |0-3|=3 fine - safe.

Three cheap arithmetic tests replace any board scanning.

23 Draw the state space tree concept for 4-queens.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Level 0 is the root: the empty board.

Each deeper level adds one queen in the next row.

Level 1 has 4 children: the first-row queen in columns 1, 2, 3, 4.

Level 2: under each level-1 node, only columns not attacked by the first queen become children - most branches are already pruned.

For example, a queen at (1,1) attacks column 1 and the diagonal through column 2 of row 2, leaving few children.

Levels 3 and 4 continue the same way; branches die whenever a row has no safe column, triggering a backtrack.

Only the paths that place all 4 queens safely reach level 4 - exactly 2 leaves, which are the 2 solutions.

The tree visualises pruning: the vast majority of the 4^4 theoretical nodes never get created.

TOPIC 9

Sum of Subsets Problem

Given n numbers and target sum m, find all subsets whose elements add to m.

Build subsets element by element (include or skip); prune when partial sum exceeds m.

Sort the numbers first; bounding uses remaining sum to prune hopeless branches.

QUESTION BANK

24 Explain the sum of subsets problem with backtracking.

IMPORTANT

Problem: given n numbers (weights) and a target m, find all subsets whose elements sum to exactly m.

Sort the weights first; then decide for each weight in order: include it or skip it.

Pseudocode with pruning:

```
backtrack(k, cur, rem): # k = next index, cur = sum so far, rem = sum of
weights k..n
if cur == m: record the subset; return
if k > n or cur > m: return # overshoot: prune
if cur + rem < m: return # cannot reach m: prune
backtrack(k+1, cur + w[k], rem - w[k]) # include w[k]
backtrack(k+1, cur, rem - w[k]) # skip w[k]
```

Trace: weights [2, 3, 5], target 5.

Include 2, include 3: cur = 5, record {2,3}.

Backtrack, skip 3: cur = 2, include 5 gives cur = 7 > 5, prune.

The remaining branches yield {5}.

The two bounding tests - overshoot and cannot-reach - kill dead branches early.

Complexity is O(2^n) worst case, but sorting plus bounding prunes most of the tree in practice.

Include-or-skip plus two bounds is the complete backtracking solution.

25 What bounding functions prune the sum-of-subsets tree?

IMPORTANT

Bounding functions prove a branch is hopeless so it can be pruned without exploration.

Upper bound (overshoot): if the current partial sum already exceeds m, no extension can fix it - prune the branch.

Lower bound (unreachable): if the current sum plus the sum of ALL remaining weights is still below m, the target is unreachable - prune.

In the fixed-tuple formulation these become precise tests at each node k:

```
include w[k+1] only if cur + w[k+1] <= m # fits without overshoot
skip w[k+1] only if cur + remaining - w[k+1] >= m # target still reachable
```

Both bounds need the weights sorted and the remaining sums precomputed.

One bound cuts branches that are too big, the other cuts branches that are too small.

26 What is the time complexity of sum of subsets?

IMPORTANT

In the worst case the algorithm explores the include-or-skip choice for every weight: 2^n subsets, so O(2^n) time.

Pruning with sorted weights and the two bounding functions cuts typical searches dramatically, but the worst case stands.

The problem is NP-complete: no known algorithm solves all instances in polynomial time.

Space is O(n) for the recursion stack plus the current subset - the tree itself is never stored.

This exponential nature is exactly why backtracking with pruning, rather than brute force, is taught for this problem.

Standard exam answer: O(2^n) worst-case time, NP-complete, O(n) space.

TOPIC 10

Hamiltonian Circuit Problem

10

A Hamiltonian circuit visits every vertex exactly once and returns to the start.

Backtracking: build a path vertex by vertex; next vertex must be adjacent and unvisited.

If stuck before covering all vertices, backtrack; a full path that connects back to start is the answer.

QUESTION BANK

27 Explain the Hamiltonian circuit problem and its backtracking solution.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

A Hamiltonian circuit is a cycle that visits every vertex exactly once and returns to the start vertex.

Backtracking builds the path vertex by vertex; each new vertex must be adjacent to the last one and not visited before.

Pseudocode:

```
path = [start]
extend():
if len(path) == n:
if edge from path[-1] to start exists: circuit found
return
```

```
for each neighbour v of path[-1]:
if v not in path: append v; extend(); remove v # backtrack
```

Trace on square A-B-C-D-A: path grows A -> B -> C -> D; D connects back to A, so A-B-C-D-A is a Hamiltonian circuit.

If a dead end is hit (no unvisited neighbour and the path is incomplete), the last vertex is removed and the next neighbour is tried.

Worst case is O(n!) but adjacency pruning - only real neighbours are ever tried - keeps small graphs practical.

The problem is NP-complete, so exponential search is expected rather than a flaw.

Grow the path, demand adjacency, backtrack on dead ends, close the cycle at the end.

28 Differentiate Hamiltonian circuit from Euler circuit.

IMPORTANT

Hamiltonian circuit: visits every VERTEX exactly once, then returns to the start; edges may be skipped.

Euler circuit: uses every EDGE exactly once and returns to the start; vertices may repeat.

Euler has a simple test: a connected graph has an Euler circuit if and only if every vertex has even degree.

Hamiltonian has no such easy test; it is NP-complete and needs backtracking search.

Example: the square A-B-C-D-A is both Hamiltonian (visits A, B, C, D) and Eulerian (uses all 4 edges; every degree is 2).

Add a diagonal A-C: degrees become A:3, C:3 - odd, so no Euler circuit exists; but A-B-C-D-A remains Hamiltonian.

Euler cares about edges and has a degree test; Hamilton cares about vertices and has no shortcut.

29 What pruning is possible in Hamiltonian circuit search?

IMPORTANT

Only neighbours of the current last vertex are ever tried - non-adjacent vertices are pruned by construction.

Vertices already in the path are skipped, which enforces the exactly-once rule automatically.

If the unvisited vertices cannot form a connected chain back to the start, the branch is abandoned early.

Fixing the start vertex avoids rediscovering the same circuit n times in rotated form.

A degree check helps: a vertex (other than the start) with fewer than 2 unused incident edges can never belong to a circuit.

Despite all this, the worst case stays exponential - pruning shrinks the constant, not the complexity class.

Adjacency plus visited-set plus start-fixing is the standard pruning package for this problem.